

The B-Spline TISE Solver

Computing bound and continuum states of a one-dimensional potential

This is a guide to the program: how to build it, how to describe a potential to it, how to read what it produces, and how to choose the parameters that control accuracy. It assumes you know the quantum mechanics and have never seen this codebase. Sections 2–5 take about twenty minutes and end with a solved problem and a plot.

Contents

1	What the program computes	2
2	Installing and building	2
3	Running a calculation	3
4	The input file	3
4.1	Describing the potential	3
4.2	Scanning a parameter	4
5	Reading the output	4
6	Choosing the basis and the box	5
6.1	Box size	5
6.2	Resolution	6
6.3	n_pts_eigenstate	6
7	Evolving a state in time	6
7.1	The recipe	6
7.2	Checking it	7
7.3	Quantities computed from $\psi(x, t)$	7
8	Worked potentials	8
9	Potentials with steps, kinks and singularities	10
9.1	Kinetic energy at a kink	10
10	δ-function potentials	10
11	Continuum states and phase shifts	12
12	Checking a result	12

1 What the program computes

Given a potential $V(x)$ that you write in a configuration file, the solver finds the eigenstates of

$$-\frac{\hbar^2}{2m} \psi''(x) + V(x) \psi(x) = E \psi(x), \quad x \in [x_{\min}, x_{\max}]. \quad (1)$$

It expands ψ in a B-spline basis $\{B_j\}$, $\psi(x) = \sum_j c_j B_j(x)$, turning (1) into the generalized matrix eigenproblem

$$H\mathbf{c} = E S \mathbf{c}, \quad H_{ij} = \frac{\hbar^2}{2m} \int B'_i B'_j dx + \int B_i V B_j dx, \quad S_{ij} = \int B_i B_j dx. \quad (2)$$

You get every eigenvalue E_n and eigenfunction $\psi_n(x)$, tabulated on a grid of your choosing, and — on request — continuum (scattering) states with their phase shifts $\delta(E)$. The potential may be piecewise, discontinuous, singular, or carry Dirac δ terms; the basis adapts to each of these on its own (Sections 9 and 10).

The wavefunction vanishes at both ends of the box, $\psi(x_{\min}) = \psi(x_{\max}) = 0$. Place the walls beyond the point where the states you care about have decayed, and they will not affect your answer; Section 6 makes that quantitative.

Time evolution is done from these eigenstates, by giving each one its phase. That calculation is about ten lines of Python and is the subject of Section 7.

2 Installing and building

You need a C++17 compiler, CMake ≥ 3.20 , Git, and Python 3.9 or newer. Every C++ library the solver uses is located on your system or, failing that, downloaded and compiled for you during the CMake configure step. The commands below are the same on macOS, Linux and Windows.

```
git clone <repository-url>
cd BSpline-CPP/1D-QM-Playground

cmake -S TISE -B TISE/build -DCMAKE_BUILD_TYPE=Release -DBUILD_TESTING=ON
cmake --build TISE/build --config Release

pip install -r requirements.txt
```

The first configure fetches dependencies, so it needs a network connection and takes a couple of minutes; later builds reuse them.

Pass both `-DCMAKE_BUILD_TYPE=Release` and `--config Release`, as above. Single-config generators (Unix Makefiles, Ninja) read the first; multi-config generators (Visual Studio, Xcode) read the second. Giving both produces an optimized build everywhere.

On Windows, run them in a “Developer Command Prompt for VS”, or any shell with MSVC on the PATH. The executable appears at `TISE/build/Release/tise_solver.exe` and the Python front end finds it for you. Building inside WSL works too, and is then the Linux case.

Confirm the installation:

```
ctest --test-dir TISE/build --build-config Release
python3 controller.py --config examples/confining-well.yaml
```

The first reports that all test suites pass. The second solves a finite square well and writes results into `data/confining-well/`.

3 Running a calculation

Drive everything through the Python front end:

```
python3 controller.py --config my-problem.yaml
```

which runs the solver and then `analysis.py` to produce plots. For the data files alone, call the binary directly:

```
./TISE/build/tise_solver --config my-problem.yaml --output-dir out/
```

A calculation is described entirely by its configuration file. Copy `examples/confining-well.yaml` and edit it.

4 The input file

```
run:
  run_tise: true
  run_tdse: false
  run_analysis: true # also produce plots
  output_dir: "./data/my-problem"

physics:
  mass: 1.0 # m in equation (1)
  hbar: 1.0 # atomic units

bspline:
  n_nodes: 201 # resolution
  order: 12 # B-spline order k
  domain: [-10.0, 10.0] # the box [x_min, x_max]

potential:
  - '{"domain': '[-10, 10]', 'function': '0.5*x^2'}"

# potential_deltas: # Dirac delta terms; see section 10
# - {x: 0.0, strength: -1.0}

tise:
  n_pts_eigenstate: 2001 # points at which each psi_n is tabulated
  error_threshold: 1.0e-10
  continuum:
    enabled: false

visualization:
  spectrum_overview: true # V(x) with each psi_n drawn on its level E_n
  eigenstates: true
```

Listing 1: A complete input file.

4.1 Describing the potential

`potential` is a list of pieces that together tile `bspline.domain`. Each piece is a string holding a small dictionary:

- **domain** — interval notation. Square brackets are closed, round brackets open, so `'[0, 5)'` means $0 \leq x < 5$. `inf` is available, as in `'(0, inf)'`.
- **function** — an expression in `x`, evaluated at run time. You have `+`, `-`, `*`, `/`, `^`, `sqrt`, `exp`, `log`, `sin`, `cos`, `tan`, `abs` and the usual companions.

A square well with a wall at the origin, in two pieces:

```
potential:
- "{ 'domain': '[0, 10)', 'function': '-1.0' }"
- "{ 'domain': '[10, 100]', 'function': '0' }"
```

Write every constant in **function** as a literal number. The expression is evaluated on its own, so `mass`, `hbar` and the other configuration fields are not in scope: for a mass-dependent coefficient, work the number out and write it in. When you change `physics.mass`, revisit any constant in **function** that depended on it.

For a fixed- ℓ radial problem, include the centrifugal term in the expression. Hydrogen's $V_\ell(r) = \ell(\ell+1)/2r^2 - 1/r$ at $\ell = 1$ becomes `'-1/x + 1/x^2'`.

Dirac δ terms have their own `potential_deltas` section, since a δ is not an expression in x (Section 10). They combine freely with whatever smooth **potential** you write.

4.2 Scanning a parameter

To vary a parameter, generate the configuration files:

```
import subprocess, yaml, pathlib

base = yaml.safe_load(open("examples/confining-well.yaml"))
for depth in [0.5, 1.0, 2.0, 4.0]:
    cfg = dict(base)
    cfg["potential"] = [
        "{ 'domain': '[0, 10)', 'function': '%g' }" % (-depth),
        "{ 'domain': '[10, 100]', 'function': '0' }",
    ]
    cfg["run"]["output_dir"] = f"./data/scan/depth{depth}"
    path = pathlib.Path(f"cfg_depth{depth}.yaml")
    path.write_text(yaml.safe_dump(cfg))
    subprocess.run(["python3", "controller.py", "--config", str(path)], check=True)
```

5 Reading the output

Results land in `<output_dir>/tise/`.

File	Contents
<code>eigenvalues.dat</code>	two columns: index (0-based), E_n
<code>eigenstates.dat</code>	column 1 is x ; column $n+2$ is $\psi_n(x)$. All eigenvalues are repeated in the <code>#</code> header
<code>continuum_states.dat</code>	the same layout, for scattering states
<code>phase_shifts.dat</code>	three columns: E , $\delta(E)$, $d\delta/dE$
<code>potential.dat</code>	x , $V(x)$ on the same grid
<code>warnings.json</code>	the solver's physics notes on the run

Loading them takes one line each:

```
import numpy as np
d = np.loadtxt("data/my-problem/tise/eigenstates.dat")
E = np.loadtxt("data/my-problem/tise/eigenvalues.dat")[:, 1]
x = d[:, 0]
phi = d[:, 1:] # phi[:, n] is psi_n(x)
```

The tabulated ψ_n come out normalized: $\int |\psi_n|^2 dx = 1$ and $\int \psi_m \psi_n dx = \delta_{mn}$ to machine precision on the output grid. Use them in quadrature directly.

`eigenvalues.dat` lists every state the basis produced. The bound states are those with $E < V(\infty)$; the ones above that threshold are the finite box's discretization of the continuum, and Section 11 is how to treat scattering properly.

The per-state files `eigenstate_NNN.dat` are numbered from 1, while the index column of `eigenvalues.dat` starts at 0. Working from `eigenstates.dat` keeps one convention throughout.

Read `warnings.json` after each run. It carries the solver's physics observations — a state reaching the outer wall, a continuum energy close to a box state, a basis coarser than the energy range requested — and each one points at a parameter worth adjusting.

6 Choosing the basis and the box

Three parameters control accuracy.

`bspline.domain` — **the box** $[x_{\min}, x_{\max}]$.

Place the walls beyond the classical turning point of the highest state you need, so ψ has decayed before reaching them.

`bspline.n_nodes` — **resolution**.

The principal knob. Double it and re-run to confirm the answer has settled.

`bspline.order` — **the B-spline order** k .

Each basis function is a piecewise polynomial of degree $k - 1$. Use 8–12. Higher order converges faster per node and widens the matrix band.

6.1 Box size

Hydrogen makes the requirement concrete. Holding `n_nodes` = 201 fixed and enlarging only the box, against $E_n = -1/2n^2$:

$R = x_{\max}$	$n = 1$	$n = 2$	$n = 3$	$n = 4$
20	3×10^{-14}	1×10^{-5}	6×10^{-3}	5×10^{-2}
40	2×10^{-15}	5×10^{-13}	1×10^{-6}	7×10^{-4}
80	6×10^{-15}	7×10^{-15}	2×10^{-15}	1×10^{-9}
160	8×10^{-15}	9×10^{-16}	2×10^{-15}	2×10^{-16}

The pattern follows the classical turning point: hydrogenic state n extends to $r \approx 2n^2$, so $n = 4$ reaches $r \approx 32$ and only starts to decay beyond it. Choose R from the highest state you intend to use — for four hydrogen states, $R \approx 160$ delivers all of them to machine precision.

A state that reaches the wall is reported in `warnings.json`:

```
bound state 6 (E=-0.00965) appears to be colliding with the outer wall at
x=100 (psi'(rMax)=0.0104, exceeds tolerance) -- its energy and wavefunction
may be inaccurate due to box truncation.
```

Enlarging the domain clears it.

6.2 Resolution

Bound-state energies converge quickly. Phase shifts ask for more, because at higher E the wavefunction oscillates faster and the node spacing has to resolve it. For a square well of depth 1 and width 10 on $[0, 100]$ at order 12, the difference in $\delta(E)$ from the closed form:

n_nodes	$E = 0.1$	$E = 0.3$	$E = 0.5$
51	3×10^{-6}	2×10^{-5}	2×10^{-4}
101	2×10^{-11}	2×10^{-12}	5×10^{-9}
201	1×10^{-13}	2×10^{-13}	6×10^{-13}
401	6×10^{-13}	4×10^{-13}	1×10^{-12}

At 51 nodes the bound states already reach 10^{-13} while $\delta(E=0.5)$ reaches 10^{-4} ; 201 nodes brings the whole range to 10^{-13} . When you are working with the continuum, take $\delta(E)$ as the quantity to converge. The solver estimates the resolvable energy ceiling from your node spacing and records it in `warnings.json`.

6.3 n_pts_eigenstate

This sets how finely the resulting ψ_n are tabulated, independently of the solve. Everything in Section 7 is quadrature on that grid, so set it generously — 2001 is a good default.

7 Evolving a state in time

A state built from eigenstates of a time-independent Hamiltonian evolves by phases alone. Writing

$$|\psi(0)\rangle = \sum_n c_n |\phi_n\rangle, \quad c_n = \langle \phi_n | \psi(0) \rangle, \quad (3)$$

and using $\hat{H}|\phi_n\rangle = E_n|\phi_n\rangle$,

$$\psi(x, t) = \sum_n c_n \phi_n(x) e^{-iE_n t/\hbar}. \quad (4)$$

This is exact: there is no time step to choose and no propagation error to control. The solver supplies $\phi_n(x)$ in `eigenstates.dat` and E_n in `eigenvalues.dat`; the rest is two integrals and a matrix product.

7.1 The recipe

The tabulated eigenstates are orthonormal on the output grid, so the overlap integrals are ordinary quadrature.

```

import numpy as np

def load(run_dir):
    """Return x, phi (columns are phi_n), and E."""
    d = np.loadtxt(f"{run_dir}/eigenstates.dat")
    E = np.loadtxt(f"{run_dir}/eigenvalues.dat")[:, 1]
    return d[:, 0], d[:, 1:], E

def coefficients(x, phi, psi0, n_states):
    """c_n = <phi_n | psi(0)>, by quadrature on the output grid."""
    return np.array([np.trapezoid(phi[:, n].conj() * psi0, x)
                     for n in range(n_states)])

def propagate(x, phi, E, c, t, hbar=1.0):
    """psi(x, t) = sum_n c_n phi_n(x) exp(-i E_n t / hbar)."""
    n = len(c)
    return phi[:, :n] @ (c * np.exp(-1j * E[:n] * t / hbar))

```

Listing 2: A complete spectral-propagation helper.

`np.trapezoid` arrived in NumPy 2.0; on NumPy 1.x the same function is `np.trapz`. To work with either, begin with

```
trapezoid = getattr(np, "trapezoid", None) or np.trapz
```

and call `trapezoid(...)` throughout.

Choose the number of states N so that $\sum_{n < N} |c_n|^2$ reaches 1 to the precision you want, using energies that are themselves converged (Section 6). Printing that sum tells you at a glance whether the basis represents your initial state.

The same `coefficients()` call gives the overlap of the eigenstates with any function you supply, which is how you project an arbitrary wavepacket — a shifted Gaussian, say — onto the spectrum.

7.2 Checking it

A coherent state in a harmonic well gives an exact benchmark, since it follows the classical trajectory. With $V = x^2/2$, $m = 1$ on $[-10, 10]$ and $\psi(x, 0) = \pi^{-1/4} e^{ik_0 x} e^{-(x-x_0)^2/2}$ at $x_0 = 2$, $k_0 = 1$, theory gives $\langle x \rangle(t) = x_0 \cos t + k_0 \sin t$. Using $N = 40$ states and the code above:

t	$\langle x \rangle$ numerical	analytic	difference
0	2.000000000	2.000000000	4×10^{-16}
0.5	2.234590662	2.234590662	4×10^{-15}
$\pi/2$	1.000000000	1.000000000	1×10^{-14}
π	-2.000000000	-2.000000000	2×10^{-14}
2π	2.000000000	2.000000000	7×10^{-15}
10	-2.222164169	-2.222164169	1×10^{-14}

with $\sum_n |c_n|^2 = 1.000000000000$.

7.3 Quantities computed from $\psi(x, t)$

```

rho = np.abs(psi_t)**2
norm = np.trapezoid(rho, x) # stays 1
xbar = np.trapezoid(rho * x, x) / norm # <x>(t)
P_ab = np.trapezoid(rho[(x>=a)&(x<=b)], x[(x>=a)&(x<=b)]) # P_[a,b](t)
pops = np.abs(c)**2 # p_n, constant in time
surv = np.abs(np.sum(np.abs(c)**2 * np.exp(-1j*E[:len(c)]*t))**2 # |<psi(0)|psi(t)>|^2

```

The survival probability $|\langle\psi(0)|\psi(t)\rangle|^2$ is the natural observable for tunneling and decay times.

8 Worked potentials

Five potentials that exercise different parts of the input language, with the accuracy each reaches.

Harmonic oscillator

```

physics: {mass: 1.0, hbar: 1.0}
bspline: {n_nodes: 151, order: 12, domain: [-10.0, 10.0]}
potential:
- "{ 'domain': '[-10, 10]', 'function': '0.5*x^2' }"

```

Gives $E_n = n + \frac{1}{2}$ to 10^{-14} . Changing `mass` to 100 gives $E_n = 0.1(n + \frac{1}{2})$ to the same accuracy, since $\omega = \sqrt{k/m}$ and the kinetic term carries the mass.

Quartic double well

```

physics: {mass: 1.0, hbar: 1.0}
bspline: {n_nodes: 201, order: 12, domain: [-10.0, 10.0]}
potential:
- "{ 'domain': '[-10, 10]', 'function': '(1/200)*x^4 - 0.25*x^2 + 1/(64*(1/200))' }"

```

With $V = \alpha x^4 - x^2/4 + 1/64\alpha$ at $\alpha = 1/200$, the minima sit at $x = \pm 5$ with $V = 0$ and the central barrier is $V(0) = 3.125$. The tunneling doublets come out directly:

doublet	E (lower)	E (upper)	splitting
1	0.48949752	0.48949813	6.1×10^{-7}
2	1.42183914	1.42193174	9.3×10^{-5}
3	2.26552000	2.27064616	5.1×10^{-3}

The splitting grows by roughly two orders of magnitude per doublet as the states approach the barrier top, and the tunneling time follows as $T = 2\pi\hbar/\Delta E$. A superposition of a doublet, localized in one well, is built by taking $(\phi_1 \pm \phi_2)/\sqrt{2}$ with the sign that matches the parity of the states you obtained.

Morse potential

```

physics: {mass: 100.0, hbar: 1.0}
bspline: {n_nodes: 201, order: 12, domain: [0.0, 10.0]}
potential:
- "{ 'domain': '[0, 10]', 'function': '(1-exp(-(x-1)))^2' }"

```

A hard wall at $x = 0$ needs nothing beyond starting the domain there, since $\psi(x_{\min}) = 0$ already. Against the analytic Morse spectrum

$$E_n = \omega_0 \left(n + \frac{1}{2}\right) - \frac{[\omega_0 \left(n + \frac{1}{2}\right)]^2}{4D_e}, \quad \omega_0 = a\sqrt{2D_e/m},$$

with $D_e = 1$, $a = 1$, $x_e = 1$, $m = 100$:

n	numerical	analytic	difference
0	0.0694606784	0.0694606781	3×10^{-10}
1	0.2008820373	0.2008820344	3×10^{-9}
2	0.3223034063	0.3223033906	2×10^{-8}
3	0.4337248047	0.4337247468	6×10^{-8}

The small difference is physical: the analytic formula describes the Morse potential on the whole line, while this calculation has a wall at $x = 0$.

Avoided crossing of two diabatic curves

A potential defined as an eigenvalue of a 2×2 matrix has a closed form,

$$V_{\pm}(x) = \frac{E_1 + E_2}{2} \pm \sqrt{\left(\frac{E_1 - E_2}{2}\right)^2 + W^2}, \quad (5)$$

so it fits in a single expression. For a Morse curve E_1 crossing a Coulombic one $E_2 = 1/x + \Delta$ with coupling $W = \alpha x/(x^2 + 1)$, at $\Delta = 0.5$ and $\alpha = 0.3$, the lower branch is

```
physics: {mass: 100.0, hbar: 1.0}
bspline: {n_nodes: 301, order: 12, domain: [0.0, 100.0]}
potential:
- "{ 'domain': '(0, 100]', 'function': '0.5*((1-exp(-(x-1)))^2 + 1/x + 0.5) - sqrt
    (0.25*((1-exp(-(x-1)))^2 - 1/x - 0.5)^2 + (0.3*x/(x^2+1))^2) }"
```

whose lowest states come out at 0.0543, 0.1852, 0.3059, 0.4193, 0.5109, 0.5116. The near-degenerate pair at the top is a clustering in the density of states — the signature of quasi-bound states behind the barrier.

To isolate such a state, multiply the tabulated wavefunction by a smooth step $f(x)$ centred on the barrier, project the result onto the spectrum with `coefficients()`, normalize, and propagate. Its survival probability then shows the decay.

Hydrogen

```
physics: {mass: 1.0, hbar: 1.0}
bspline: {n_nodes: 201, order: 12, domain: [0.0, 160.0]}
potential:
# l = 0:
- "{ 'domain': '(0, 160]', 'function': '-1/x' }"
# l = 1 instead:
# - "{ 'domain': '(0, 160]', 'function': '-1/x + 1/x^2' }"
```

Equation (1) is already the reduced radial equation for $u_{n\ell}(r)$, so the ψ in the output is $u_{n\ell}$ — use it as it stands, without dividing by r .

At $\ell = 1$ the solver imposes $u(0) = 0$ and the centrifugal term $1/r^2$ supplies the additional suppression at the origin variationally: with $R = 100$ the states come out at -0.125 , $-0.0555\bar{5}$ and -0.03125 , with differences of 4×10^{-15} , 1×10^{-16} and 3×10^{-13} from $-1/2n^2$ for $n = 2, 3, 4$.

9 Potentials with steps, kinks and singularities

Write the potential in whatever pieces the physics calls for; the basis adapts at each junction on its own.

The adaptation is the one you would choose by hand. A B-spline basis of order k with a knot of multiplicity m is C^{k-1-m} at that knot. An ordinary grid node has $m = 1$ and so gives C^{k-2} , smoother than ψ actually is where V jumps. The solver classifies every junction between pieces and adds degenerate knots to match the smoothness ψ really has:

Junction	What jumps	ψ regularity	multiplicity	extra knots
Step	V itself	C^1 (ψ'' jumps)	$k - 2$	$k - 3$
Kink	V' only	C^2 (ψ''' jumps)	$k - 3$	$k - 4$
Singular	$V \rightarrow \infty$	—	$k - 1$	$k - 2$; the one B-spline nonzero at the point is removed

The junction is also spliced into the grid as a node. Quadrature then runs interval by interval between consecutive nodes, 64-point Gauss–Legendre on each, so every quadrature interval lies entirely on one side of the discontinuity and sees a smooth integrand.

Against the exact transcendental solution for a square well, the four bound states come out to 8×10^{-14} with the step at $x = 10$, which falls on the uniform grid, and to 2×10^{-14} with the step at $x = 10.5$, which does not. Put discontinuities wherever the physics puts them.

9.1 Kinetic energy at a kink

The kinetic matrix element is

$$T_{ij} = \frac{\hbar^2}{2m} \int B'_i(x) B'_j(x) dx, \quad (6)$$

the weak (variational) form, with the integration by parts already carried out and the boundary term vanishing under the Dirichlet conditions.

This is what lets a kink be handled exactly. Where ψ has a kink, ψ'' contains a δ -function; in (6) only first derivatives appear, and those are well defined for a C^0 function. The variational structure does the rest.

10 δ -function potentials

Declare Dirac deltas in their own section:

```
potential: # the smooth part, as always
- '{"domain': '[-20, 20]', 'function': '0'}"

potential_deltas: # adds strength * delta(x - x0)
- {x: 0.0, strength: -1.0}
```

Each entry adds $\text{strength} \cdot \delta(x - x_0)$ to the potential, with x_0 the entry's `x` field and `strength` negative for an attractive well. Place x_0 strictly inside the domain. `examples/delta-well.yaml` is ready to run.

What the solver does with them. Two things, both automatic.

First, it sets the knot multiplicity. A δ at x_0 leaves ψ continuous and makes ψ' jump, so the basis is made exactly C^0 there: multiplicity $k - 1$, that is $k - 2$ degenerate knots beyond the ordinary node, spliced in if x_0 was not already a grid point. That multiplicity is what makes the result exact:

extra knots	multiplicity	continuity	E_0	difference
0	1	C^6	-0.42090181	8×10^{-2}
2	3	C^4	-0.45229411	5×10^{-2}
4	5	C^2	-0.47977270	2×10^{-2}
$6 = k - 2$	7	C^0	-0.50000000000002	2×10^{-14}

($k = 8$, $\lambda = 1$; the last row is what the solver uses.)

Second, it adds the matrix element, which for a δ is a point evaluation rather than a quadrature:

$$\langle B_i | \lambda \delta(x - x_0) | B_j \rangle = \lambda B_i(x_0) B_j(x_0). \quad (7)$$

Nothing here evaluates ψ'' , where the δ induced by the kink would live: the kinetic term is the weak form (6), so only first derivatives appear.

Accuracy. The single attractive well $V = -\lambda\delta(x)$ has $E_0 = -\lambda^2/2$ and $\psi_0 = \sqrt{\lambda}e^{-\lambda|x|}$. On $[-20, 20]$ with `n.nodes` = 81, `order` = 8:

λ	E_0 computed	exact	difference
0.5	-0.124999998969	-0.125	1×10^{-9}
1.0	-0.500000000000	-0.5	2×10^{-14}
2.0	-1.999999999992	-2.0	8×10^{-12}

and $\psi(x)/\psi(0)$ follows $e^{-|x|}$ to eight digits. A weakly bound δ spreads further — at $\lambda = 0.5$, $e^{-0.5 \times 20} = 4.5 \times 10^{-5}$ is still appreciable at the wall — so widen the domain as λ decreases.

Several at once. Deltas accumulate, so the double well $V = -\lambda[\delta(x + a) + \delta(x - a)]$ is two entries:

```
potential_deltas:
- {x: -1.0, strength: -1.0}
- {x: 1.0, strength: -1.0}
```

producing the even/odd doublet with $\kappa = \lambda(1 \pm e^{-2\kappa a})$ and the even state lower. Deltas also combine with any smooth `potential` — a δ in a harmonic well, on a step, or on a Coulomb tail all work.

Quadrature on a cusped state. A δ makes ψ cusped, and the trapezoid rule converges as $O(h^2)$ across a cusp. On a 1001-point output grid, $\int |\psi|^2 dx$ evaluated with `np.trapezoid` reads 1.00053; the eigenvector itself is exactly normalized, and the difference is the output-grid quadrature. It scales cleanly — 5.3×10^{-4} , 1.3×10^{-4} , 3.3×10^{-5} , 8.3×10^{-6} as `n_pts_eigenstate` doubles from 1001 to 8001 — so raise `n_pts_eigenstate` to the accuracy you need.

11 Continuum states and phase shifts

Turn them on with:

```
tise:
  continuum:
    enabled: true
    E_threshold: 0.0 # bottom of the continuum, i.e. V at infinity
    E_max: 0.5
    n_energies: 50
    n_pts: 500
```

The solver builds scattering states and extracts $\delta(E)$ by matching to the asymptotic form at $x_{\max}$. Choose $x_{\max}$ in a region where V has flattened out, so the matching sees the asymptotic behaviour.

For a potential with a genuine $1/x$ Coulomb tail, declare the domain unbounded — `'(0, inf)'` rather than `'(0, 100]'` — so the tail is recognized, and set `tise.continuum.l` to the angular momentum of the $\ell(\ell+1)/2x^2$ term you wrote into `function`. The two must agree: a centrifugal term decays faster than $1/x$, so the tail fit takes ℓ from this field rather than reading it off the potential.

Validated on a square well of depth 1 and width 10 with a wall at the origin, against $\tan(ka+\delta) = (k/K)\tan(Ka)$: at `n_nodes = 201` the largest difference over 50 energies in $(0, 0.5]$ is 1.8×10^{-12} .

`n_energies` also sets how many `continuum_NNN.png` figures the analysis stage writes, so 50 energies give 50 plots.

Energies near a box state. In a finite box the continuum is represented by a dense set of discrete states, and a requested energy landing on one of them gives a state dominated by that box resonance. The solver reports this:

```
continuum energy grid point E=0.04 is within 0.00083 of confined eigenvalue
E_12=0.0408; this energy's continuum state is likely a finite-box
discretization artifact, not a physical feature
```

Shift the energy grid, or enlarge the box so the box states lie closer together.

12 Checking a result

1. Read `warnings.json` and act on what it reports.
2. Double `n_nodes`, re-run, and confirm the numbers hold.
3. Check that the highest state you use fits inside the box, from its classical turning point.
4. Select the bound states from `eigenvalues.dat` by $E < V(\infty)$.
5. For time evolution, confirm $\sum_n |c_n|^2 = 1$ and that $\int |\psi(x, t)|^2 dx$ holds at 1 across the whole time range.
6. For continuum work, converge on $\delta(E)$.
7. Compare against whatever closed form or limiting case the problem has: harmonic levels, the Morse formula, hydrogenic $-1/2n^2$, a tunneling splitting, a square-well phase shift, $-\lambda^2/2$ for a δ well.

Further reading in the repository: `docs/guides/input-file-authoring-guide.md` for the complete input-file reference; `TISE/README.md` for the build system and the numerical methods; `examples/confining-well.yaml` and `examples/delta-well.yaml` for annotated starting points.